FlexQL Design Document

Name: PERLA.HARSHAVARDHAN
Roll: 25CS60R72
GitHub: https://github.com/Harsha-585/FlexQL
Version: 1.0
Date: April 2026

Table of Contents

1. Overview
2. Data Storage
3. Indexing Method
4. Caching Strategy
5. Handling of Expiration Timestamps
6. Multithreading Design
7. Additional Design Decisions
8. Performance Optimizations
9. File Structure Reference
10. Benchmark Results & Test Results
11. Summary
12. Appendix: SQL Syntax Reference

---

1. Overview

FlexQL is a lightweight, disk-persistent SQL database implemented in C++17. It provides:

- **SQL Support**: CREATE TABLE, INSERT, SELECT (with JOIN, WHERE, ORDER BY), DELETE
- **Disk Persistence**: All data is stored on disk with a buffer pool cache
- **B+ Tree Indexing**: Automatic indexing on primary key columns
- **Expiration Support**: Built-in TTL (Time-To-Live) via EXPIRES_AT column
- **Concurrent Access**: Multi-threaded client handling with proper synchronization

Architecture Diagram:

```
┌──────────────────────────────────────────────────────────────────────┐
│                              Client (TCP)                            │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                              Server Layer                            │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────────────┐    │
│   │   Parser     │→  │  Executor    │→  │  Query Result        │    │
│   │ (Lexer+AST)  │   │ (SQL Logic)  │   │  (Serialization)     │    │
│   └──────────────┘   └──────────────┘   └──────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              Storage Engine                              │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐             │
│  │   Catalog    │   │  Buffer Pool │   │   B+ Tree Index  │             │
│  │  (Metadata)  │   │  (LRU Cache) │   │  (Primary Keys)  │             │
│  └──────┬───────┘   └──────┬───────┘   └──────┬───────────┘             │
│         │                  │                  │                         │
│  ┌──────▼──────────────────▼──────────────────▼──────────┐              │
│  │                    Pager (Disk I/O)                    │              │
│  └────────────────────────────────────────────────────────┘              │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
                     ┌──────────────────────────┐
                     │     Disk Files           │
                     │   - catalog.meta         │
                     │   - tables/*.dat         │
                     │   - indexes/*.idx        │
                     └──────────────────────────┘
```

---

2. Data Storage

2.1 File Organization

FlexQL stores data in three types of files:

File Type	Location	Purpose
Catalog	`data/catalog.meta`	Table schemas and metadata
Table Data	`data/tables/[TABLE].dat`	Row data in binary pages
Index	`data/indexes/[TABLE].idx`	B+ tree index pages

2.2 Page Structure

All disk I/O operates on fixed-size **4096-byte pages** (standard OS page size):

```
Page Layout (4096 bytes total):
┌──────────────────────────────────────────────────────────────────────────┐
│ HEADER (16 bytes)                                                        │
│  ┌──────────┬──────────┬──────────┬──────────┬──────────┐               │
│  │ Page ID  │  Type    │ # Records│ Free Space│ Next Page│               │
│  │ 4 bytes  │ 1 byte   │ 2 bytes  │  Offset  │ 4 bytes  │               │
│  │          │          │          │ 2 bytes  │          │               │
│  └──────────┴──────────┴──────────┴──────────┴──────────┘               │
├──────────────────────────────────────────────────────────────────────────┤
│ DATA AREA (variable)                                                     │
│  ┌──────────────────────────────────────────────────────────┐            │
│  │ Record 1 | Record 2 | Record 3 | ... | Free Space        │            │
│  └──────────────────────────────────────────────────────────┘            │
├──────────────────────────────────────────────────────────────────────────┤
│ SLOT DIRECTORY (grows backward from end)                                 │
```

```
│  ┌──────────────────┬──────────────────┬──────────────────────────┐ │
│  │ Slot 3: [off,len] │ Slot 2: [off,len] │ Slot 1: [off,len]  ||
│  └──────────────────┴──────────────────┴──────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
```

Page Types:

- `TABLE_DATA` (1): Contains serialized row records
- `BPTREE_INTERNAL` (2): B+ tree internal nodes
- `BPTREE_LEAF` (3): B+ tree leaf nodes
- `CATALOG` (4): Catalog metadata
- `FREE` (0): Unused page

2.3 Record Serialization

Records are stored in binary format within pages:

```
Record Binary Format:
┌──────────────────────────────────────────────────────────────────┐
│ Expiration Timestamp (8 bytes, int64_t)                          │
├──────────────────────────────────────────────────────────────────┤
│ Column 1:                                                        │
│   ├─ Null Flag (1 byte): 0=has value, 1=NULL                     │
│   └─ Value (varies by type):                                     │
│       INT/DATETIME: 8 bytes (int64_t)                            │
│       DECIMAL: 8 bytes (double)                                  │
│       VARCHAR/TEXT: 2 bytes length + string data                 │
├──────────────────────────────────────────────────────────────────┤
│ Column 2: [null flag][value]                                     │
├──────────────────────────────────────────────────────────────────┤
│ ... (additional columns)                                         │
└──────────────────────────────────────────────────────────────────┘
```

Supported Data Types:

Type	Storage Size	Description
INT	8 bytes	64-bit signed integer
DECIMAL	8 bytes	IEEE 754 double-precision float
DATETIME	8 bytes	Unix timestamp (seconds)
VARCHAR(n)	2 + n bytes	Variable-length string with max length
TEXT	2 + n bytes	Unlimited length string

2.4 Pager (Disk I/O Layer)

The `Pager` class handles all disk operations:

```
class Pager {
    std::fstream file_;       // Binary file handle
    std::mutex mutex_;        // Thread-safe file access
    uint32_t logical_pages;   // Total allocated pages
    uint32_t physical_pages;  // Pages written to disk
};
```

Key Operations:

- `read_page(page_id)`: Load page from disk into memory
- `write_page(page_id, data)`: Persist page to disk
- `allocate_page()`: Reserve new page (extends file by 64 pages at a time)

Optimization: File pre-extension allocates 64 pages (256KB) at once to reduce syscall overhead.

---

3. Indexing Method

3.1 B+ Tree Structure

FlexQL uses a **B+ Tree** for primary key indexing with order **128**:

```
B+ Tree Configuration:
- Order: 128
- Page Size: 4096 bytes
- Max Leaf Entries: ~291 per leaf node
- Max Internal Entries: ~340 per internal node
- Key Type: double (8 bytes, supports INT/DECIMAL)
```

Why B+ Tree?

1. **Efficient Range Queries**: Leaf nodes are linked, enabling O(log n + k) range scans
2. **Disk Optimization**: High fanout minimizes tree height and disk seeks
3. **Sorted Access**: Supports ORDER BY on primary key efficiently

3.2 Node Structures

Leaf Node Layout:

```
┌──────────────────────────────────────────────────────────────────┐
│ Header (12 bytes)                                                │
│ ┌──────────┬──────────┬──────────┬──────────────────────────┐   │
│ │ Page ID  │ Type (1) │ # Keys   │ Parent Page ID           │   │
│ │ 4 bytes  │ 1 byte   │ 2 bytes  │ 4 bytes                  │   │
│ └──────────┴──────────┴──────────┴──────────────────────────┘   │
├──────────────────────────────────────────────────────────────────┤
│ Leaf Metadata (8 bytes)                                          │
│ ┌──────────────────────┬──────────────────────┐                  │
│ │ Next Leaf Page ID    │ Prev Leaf Page ID    │                  │
│ │ 4 bytes              │ 4 bytes              │                  │
│ └──────────────────────┴──────────────────────┘                  │
├──────────────────────────────────────────────────────────────────┤
│ Entries (14 bytes each × up to 291)                              │
│ ┌──────────┬──────────┬──────────┐                               │
│ │ Key (8B) │ Page ID (4B)│ Slot (2B) │  → RecordPtr              │
│ └──────────┴──────────┴──────────┘                               │
│ ... (sorted by key)                                              │
└──────────────────────────────────────────────────────────────────┘
```

Internal Node Layout:

```
┌──────────────────────────────────────────────────────────────┐
│ Header (12 bytes) + First Child Pointer (4 bytes)            │
├──────────────────────────────────────────────────────────────┤
│ Entries (12 bytes each × up to 340)                          │
│  ┌──────────────────────┬──────────────────────┐             │
│  │ Key (8 bytes)        │ Child Page ID (4B)   │             │
│  └──────────────────────┴──────────────────────┘             │
│ ... (sorted by key)                                          │
└──────────────────────────────────────────────────────────────┘
```

3.3 Index Operations

Operation	Time Complexity	Description
`insert(key, ptr)`	O(log n)	Insert with automatic leaf split
`search(key)`	O(log n)	Exact key lookup
`range_scan(low, high)`	O(log n + k)	Range query, returns k matches
`full_scan()`	O(n)	Iterate all entries via leaf links
`remove(key)`	O(log n)	Simple removal (no rebalancing)

Automatic Indexing: Primary key columns are automatically indexed when table is created.

---

4. Caching Strategy

4.1 Buffer Pool Architecture

FlexQL implements an **LRU (Least Recently Used) Buffer Pool**:

```
class BufferPool {
    unordered_map<uint64_t, CacheEntry> pages_;      // All cached pages
    list<uint64_t> unpinned_list_;                   // LRU tracking (un
    unordered_map<uint64_t, iterator> unpinned_map_;// O(1) LRU positio
    size_t capacity_ = 262144;                       // 1GB (262144 × 4K
};

struct CacheEntry {
    Page page;            // The cached page data
    Pager* owner;         // Which file this page belongs to
    bool dirty;           // Modified since load?
    int pin_count;        // References holding this page
};
```

4.2 Cache Operations

Page Fetch (`fetch_page`):

```
1. Compute key = (pager_ptr << 32) | page_id
2. If key in cache:
   a. Increment pin_count
```

```
   b. If was unpinned, remove from unpinned_list (move to "in use")
   c. Return cached page
3. Else (cache miss):
   a. If cache full and no unpinned pages → grow capacity
   b. If cache full and has unpinned → evict LRU (back of list)
   c. Read page from disk via pager
   d. Add to cache with pin_count = 1
   e. Return page
```

Page Unpin (`unpin`):

```
1. Decrement pin_count
2. If pin_count == 0:
   a. Add to front of unpinned_list (MRU position)
   b. Page is now evictable
```

Eviction (`evict_one`):

```
1. Take from back of unpinned_list (LRU)
2. If dirty: write to disk
3. Remove from cache
4. Return freed slot
```

4.3 Design Rationale

Decision	Rationale
1GB Default Capacity	Holds ~262K pages, sufficient for 10M+ rows
LRU Eviction	Adapts to working set, keeps hot pages in memory
O(1) Operations	Hash map + doubly-linked list for all operations
Pin Counting	Prevents eviction of in-use pages
Dirty Tracking	Only write modified pages to disk
Composite Keys	Single pool shared across all tables

---

5. Handling of Expiration Timestamps

5.1 Storage

Every record stores an 8-byte expiration timestamp as the **first field**:

```
struct Record {
    int64_t expiration_ts;    // 0 = never expires
    std::vector<Value> values;

    bool is_expired() const {
        if (expiration_ts <= 0) return false;
        return std::time(nullptr) > expiration_ts;
    }
};
```

5.2 Setting Expiration

Expiration is set automatically when a table has an `EXPIRES_AT` column:

```
CREATE TABLE sessions (
    session_id INT PRIMARY KEY,
    user_id INT,
    EXPIRES_AT DATETIME    -- Special column name
);

INSERT INTO sessions VALUES (1, 100, 1712534400);  -- Expires at timest
```

Detection Logic (in `execute_insert`):

```
// Find EXPIRES_AT column index
for (int i = 0; i < columns.size(); i++) {
    if (columns[i].name == "EXPIRES_AT") {
        expires_at_idx = i;
    }
}

// Extract timestamp during insert
if (expires_at_idx >= 0) {
    record.expiration_ts = (int64_t)row[expires_at_idx].to_double();
}
```

5.3 Expiration Checking

Current Behavior: Records are stored with expiration metadata but **not automatically filtered** during queries. This is by design:

- **Application Layer Filtering**: Caller can check `is_expired()` on results
- **Soft Delete Semantics**: Expired records remain until explicit DELETE
- **Audit Trail**: Enables "show recently expired" queries

To Enable Auto-Filtering (if needed):

```
// In scan_table(), add filter:
for (const Record& record : records) {
    if (!record.is_expired()) {
        results.push_back(record);
    }
}
```

5.4 Expiration Format

Value	Meaning
`0` or negative	Never expires
Positive integer	Unix timestamp (seconds since 1970)

---

6. Multithreading Design

6.1 Threading Model

FlexQL uses a **thread-per-client** model:

```
Main Thread                    Worker Threads
────────────                   ──────────────
    │
    ▼
accept() loop ──────────────► Thread 1 → Client 1
    │                         ► Thread 2 → Client 2
    │                         ► Thread 3 → Client 3
    ▼                            ...
(continues accepting)
```

Thread Pool Size: Configured as 8 in types.h (not enforced - unlimited threads)

6.2 Synchronization Hierarchy

Locks are acquired in this order to prevent deadlocks:

```
Level 1: Table Mutex (coarse-grained)
    │
    ├── Level 2: Index Mutex (B+ tree operations)
    │       │
    │       └── Level 3: Buffer Pool Mutex (page operations)
    │               │
    │               └── Level 4: Pager Mutex (disk I/O)
    │
    └── Level 3: Buffer Pool Mutex (direct page access)
```

6.3 Lock Types and Scope

Component	Lock Type	Scope	Purpose
`TableInfo::rw_mutex`	`std::mutex`	Per-table	Protects table data during queries
`BPlusTree::mutex_`	`std::recursive_mutex`	Per-index	Allows batch inserts (same thread can re-lock)
`BufferPool::mutex_`	`std::mutex`	Global	Serializes all cache operations
`Pager::mutex_`	`std::mutex`	Per-file	Serializes disk read/write
`Catalog::catalog_mutex_`	`std::mutex`	Global	Protects table creation/deletion

6.4 Batch Operation Optimization

For bulk inserts, B+ tree lock is held across the entire batch:

```
// In execute_insert():
if (has_primary_key) {
    table->index->lock();        // Lock once
}
```

```
for (const auto& row : batch) {
    insert_record(...);
    insert_into_index_unlocked(...);  // No per-row locking
}

if (has_primary_key) {
    table->index->unlock();      // Unlock once
}
```

This reduces lock acquisition from O(n) to O(1) for n-row batches.

6.5 Socket Configuration

Network connections are optimized for throughput:

```
// Disable Nagle's algorithm (reduce latency)
setsockopt(socket, IPPROTO_TCP, TCP_NODELAY, &flag, sizeof(flag));

// Large buffers for batch operations
int bufsize = 1048576;  // 1MB
setsockopt(socket, SOL_SOCKET, SO_SNDBUF, &bufsize, sizeof(bufsize));
setsockopt(socket, SOL_SOCKET, SO_RCVBUF, &bufsize, sizeof(bufsize));
```

---

7. Additional Design Decisions

7.1 Network Protocol

Format: Text-based SQL over TCP

Request: SQL statements terminated by `;`

```
INSERT INTO users VALUES (1, 'Alice');
SELECT * FROM users WHERE id = 1;
```

Response Format:

```
COLS    col1    col2    col3       ← Tab-separated column names
ROW     val1    val2    val3       ← Tab-separated row values
ROW     val4    val5    val6
OK                                  ← Success marker
END                                 ← End of response

ERROR: message                      ← On failure
END
```

7.2 Parser Design

Architecture: Recursive descent parser with single-token lookahead

```
SQL Input → Lexer → Token Stream → Parser → AST → Executor
```

Optimizations:

- Zero-copy input: Lexer operates on `const char*`, not copied string
- Static token strings: Punctuation tokens use pre-allocated strings
- Length-based keyword matching: Fast switch on identifier length

7.3 Query Execution

Query Type	Execution Strategy
SELECT without index	Full table scan, in-memory filtering
SELECT with PK equality	B+ tree point lookup, O(log n)
SELECT with PK range	B+ tree range scan
JOIN	Hash join on smaller table (O(n+m))
ORDER BY	In-memory quicksort
DELETE	Full table rewrite (simple, no rebalancing)

7.4 Catalog Persistence

Metadata is stored in `data/catalog.meta`:

```
Binary Format:
┌──────────────────────────────────────────┐
│ Number of Tables (4 bytes)               │
├──────────────────────────────────────────┤
│ Table 1:                                 │
│   Name Length (2B) + Name                │
│   Primary Key Index (4B)                 │
│   First/Last Data Page (4B each)         │
│   Row Count (8B)                         │
│   Column Count (2B)                      │
│   For each column:                       │
│     Name Length (2B) + Name              │
│     Type (1B)                            │
│     Max Length (2B)                      │
│     Flags (1B): NOT NULL, PRIMARY KEY    │
├──────────────────────────────────────────┤
│ Table 2: ...                             │
└──────────────────────────────────────────┘
```

7.5 RESET Command

The `RESET table_name` command efficiently clears all data:

1. Truncates data file to 0 bytes
2. Reinitializes with single empty page
3. Rebuilds empty B+ tree index
4. Resets row count to 0

---

8. Performance Optimizations

8.1 Memory Optimizations

Optimization	Impact
Pre-allocated string buffers	Reduces allocations during parsing
Move semantics for Records	Avoids copying row data
Value union for numeric types	Reduces Value struct size
Reused serialization buffer	Single allocation per batch

8.2 I/O Optimizations

Optimization	Impact
64-page file pre-extension	Reduces syscalls during growth
256KB server read buffer	Fewer read() calls for large batches
TCP_NODELAY	Lower latency for small responses
1MB socket buffers	Better throughput for bulk operations

8.3 Algorithmic Optimizations

Optimization	Impact
O(1) LRU eviction	Fast cache management
Hash join for JOINs	$O(n+m)$ vs $O(n \times m)$ nested loop
Binary search in B+ tree	O(log k) within each node
Batch index locking	Amortizes lock overhead

8.4 Compiler Optimizations

Build flags in Makefile:

```
CXXFLAGS = -std=c++17 -O3 -march=native -flto -ffast-math -funroll-loop
```

Flag	Purpose
`-O3`	Maximum optimization level
`-march=native`	CPU-specific instructions
`-flto`	Link-time optimization
`-ffast-math`	Faster floating-point operations
`-funroll-loops`	Reduce loop overhead

---

9. File Structure Reference

```
flexql/
├── include/
│   ├── common/
│   │   └── types.h          # Constants, Value, Record, Column structs
│   ├── storage/
│   │   ├── page.h           # Page layout and operations
```

```
│   │   ├── pager.h          # Disk I/O interface
│   │   ├── record.h         # Serialization interface
│   │   └── catalog.h        # Table metadata management
│   ├── index/
│   │   └── bptree.h         # B+ tree interface
│   ├── cache/
│   │   └── lru_cache.h      # Buffer pool interface
│   ├── parser/
│   │   ├── ast.h            # SQL AST structures
│   │   └── parser.h         # Lexer and Parser
│   ├── query/
│   │   └── executor.h       # Query execution engine
│   └── network/
│       └── server.h         # TCP server
├── src/
│   ├── storage/             # Storage implementations
│   ├── index/               # B+ tree implementation
│   ├── cache/               # Buffer pool implementation
│   ├── parser/              # SQL parsing
│   ├── query/               # Query execution
│   ├── server/              # Network server
│   └── client/              # Client library + REPL
├── data/                    # Runtime data (auto-created)
│   ├── catalog.meta
│   ├── tables/
│   └── indexes/
├── Makefile
└── DESIGN_DOCUMENT.md       # This file
```

---

10. Benchmark Results & Test Results

10.1 Bulk Insert Benchmark

Test Configuration:

- **Dataset**: 10,000,000 rows
- **Batch Size**: 50,000 rows per INSERT statement
- **Table Schema**: `BIG_USERS (id INT, name VARCHAR, balance DECIMAL)`
- **Hardware**: Apple Silicon / x86-64 (commodity hardware)

Results:

```
┌──────────────────────────────────────────────────────────────┐
│              BULK INSERT BENCHMARK RESULTS                   │
├──────────────────────────────────────────────────────────────┤
│  Total Rows Inserted:     10,000,000                         │
│  Total Time:              10,208 ms (~10.2 seconds)          │
│  Throughput:              979,623 rows/second                │
│  Batches Processed:       200 batches                        │
│  Average per Batch:       51 ms                              │
└──────────────────────────────────────────────────────────────┘
```

Performance Breakdown:

Phase	Time	Notes
CREATE TABLE	0 ms	Schema creation (cached)
RESET TABLE	~170 ms	Clear existing data
INSERT 10M rows	10,208 ms	Main benchmark
Total	**~10.4 seconds**	End-to-end

10.2 Query Performance Tests

Single Operations (measured on ~103 row dataset):

Query Type	Time	Description
CREATE TABLE	27 ms	Create new table with schema
INSERT (1 row)	<1 ms	Single row insertion
INSERT (2 rows)	<1 ms	Multi-value insert
INSERT (100 rows)	<1 ms	Batch insert
SELECT by PK	<1 ms	Indexed lookup (B+ tree)
SELECT with filter	<1 ms	Full scan with WHERE
SELECT all rows	<1 ms	Full table scan
SELECT ORDER BY	<1 ms	Sorted result set
INNER JOIN	<1 ms	Hash join on two tables
DELETE single row	780 ms	Table rewrite (unoptimized)

10.3 Unit Test Results

Test Suite: 22 tests covering all SQL operations

```
┌──────────────────────────────────────────────────────────────────────┐
│                        UNIT TEST RESULTS                             │
├──────────────────────────────────────────────────────────────────────┤
│  [PASS] CREATE TABLE BIG_USERS                          (0 ms)       │
│  [PASS] RESET BIG_USERS                               (170 ms)       │
│  [PASS] INSERT benchmark complete                  (10208 ms)        │
│  [PASS] CREATE TABLE TEST_USERS                         (0 ms)       │
│  [PASS] RESET TEST_USERS                              (170 ms)       │
│  [PASS] INSERT TEST_USERS                               (0 ms)       │
│  [PASS] Single-row value validation                                  │
│  [PASS] Filtered rows validation                                     │
│  [PASS] ORDER BY descending validation                               │
│  [PASS] Empty result-set validation                                  │
│  [PASS] CREATE TABLE TEST_ORDERS                        (8 ms)       │
│  [PASS] RESET TEST_ORDERS                              (36 ms)       │
│  [PASS] INSERT TEST_ORDERS                              (0 ms)       │
│  [PASS] Join result validation                                       │
│  [PASS] Single-condition equality WHERE validation                   │
│  [PASS] Join with no matches validation                              │
│  [PASS] Invalid SQL should fail                                      │
│  [PASS] Missing table should fail                                    │
```

```
├──────────────────────────────────────────────────────────────────────┤
│  SUMMARY: 22/22 passed, 0 failed                                     │
└──────────────────────────────────────────────────────────────────────┘
```

10.4 Detailed Query Examples

Example 1: Table Creation

```
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    balance DECIMAL
);
-- Result: OK (27 ms)
```

Example 2: Batch Insert

```
INSERT INTO users VALUES
    (1, 'Alice', 1000.50),
    (2, 'Bob', 2500.75),
    (3, 'Charlie', 500.00);
-- Result: OK (<1 ms)
```

Example 3: Indexed Lookup

```
SELECT * FROM users WHERE id = 1;
-- Result:
-- COLS    id    name    balance
-- ROW     1     Alice   1000.50
-- Time: <1 ms (B+ tree lookup)
```

Example 4: Filter Query

```
SELECT * FROM users WHERE balance > 500;
-- Result:
-- COLS    id    name    balance
-- ROW     1     Alice   1000.50
-- ROW     2     Bob     2500.75
-- Time: <1 ms (full scan + filter)
```

Example 5: JOIN Query

```
SELECT * FROM users
INNER JOIN orders ON users.id = orders.user_id;
-- Result:
-- COLS    id    name    balance    order_id    user_id    amount
-- ROW     1     Alice   1000.50    1           1          150.00
-- ROW     1     Alice   1000.50    2           1          250.00
-- ROW     2     Bob     2500.75    3           2          100.00
-- Time: <1 ms (hash join)
```

Example 6: Sorted Results

```
SELECT * FROM users ORDER BY balance DESC;
-- Result:
```

```
-- COLS    id    name    balance
-- ROW     2     Bob     2500.75
-- ROW     1     Alice   1000.50
-- ROW     3     Charlie 500.00
-- Time: <1 ms (quicksort)
```

10.5 Scalability Analysis

Row Count	Insert Time	Throughput	Notes
1,000	<1 ms	>1M rows/s	All in cache
10,000	~10 ms	~1M rows/s	All in cache
100,000	~100 ms	~1M rows/s	All in cache
1,000,000	~1,000 ms	~1M rows/s	Minimal disk I/O
10,000,000	~10,208 ms	~980K rows/s	Buffer pool utilized

10.6 Memory Usage

Component	Size	Configuration
Buffer Pool	1 GB max	262,144 pages × 4KB
Per-Page Overhead	16 bytes	Header
B+ Tree Node	4 KB	Fits ~291 leaf entries
Socket Buffers	1 MB each	Send + Receive

---

11. Summary

FlexQL is designed as a lightweight, embeddable SQL database with:

- **Disk-first persistence** with memory caching for performance
- **B+ tree indexing** for efficient primary key operations
- **LRU buffer pool** with 1GB default capacity
- **Built-in TTL support** via EXPIRES_AT column
- **Thread-per-client** model with hierarchical locking
- **Text-based SQL protocol** for easy debugging and interoperability

The design prioritizes simplicity and correctness while achieving **~1M rows/second** insertion throughput on commodity hardware.

---

12. Appendix: SQL Syntax Reference

Supported Statements

```
-- Table Creation
CREATE TABLE [IF NOT EXISTS] table_name (
    column1 TYPE [PRIMARY KEY],
    column2 TYPE,
    ...
);
```

```
-- Data Insertion
INSERT INTO table_name VALUES
    (val1, val2, ...),
    (val1, val2, ...);

-- Data Retrieval
SELECT * | col1, col2 FROM table_name
    [INNER JOIN table2 ON condition]
    [WHERE condition]
    [ORDER BY column [ASC|DESC]];

-- Data Deletion
DELETE FROM table_name [WHERE condition];
```

Data Types

Type	Description	Example
`INT`	64-bit integer	`42, -100`
`DECIMAL`	Double-precision float	`3.14, -99.9`
`VARCHAR(n)`	Variable string (max n)	`'Hello'`
`TEXT`	Unlimited string	`'Long text...'`
`DATETIME`	Unix timestamp	`1712534400`

Operators

Operator	Description
`=`	Equality
`>`	Greater than
`<`	Less than
`>=`	Greater or equal
`<=`	Less or equal
`AND`	Logical AND
`OR`	Logical OR

Document generated: April 2026
FlexQL Version: 1.0